

# PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

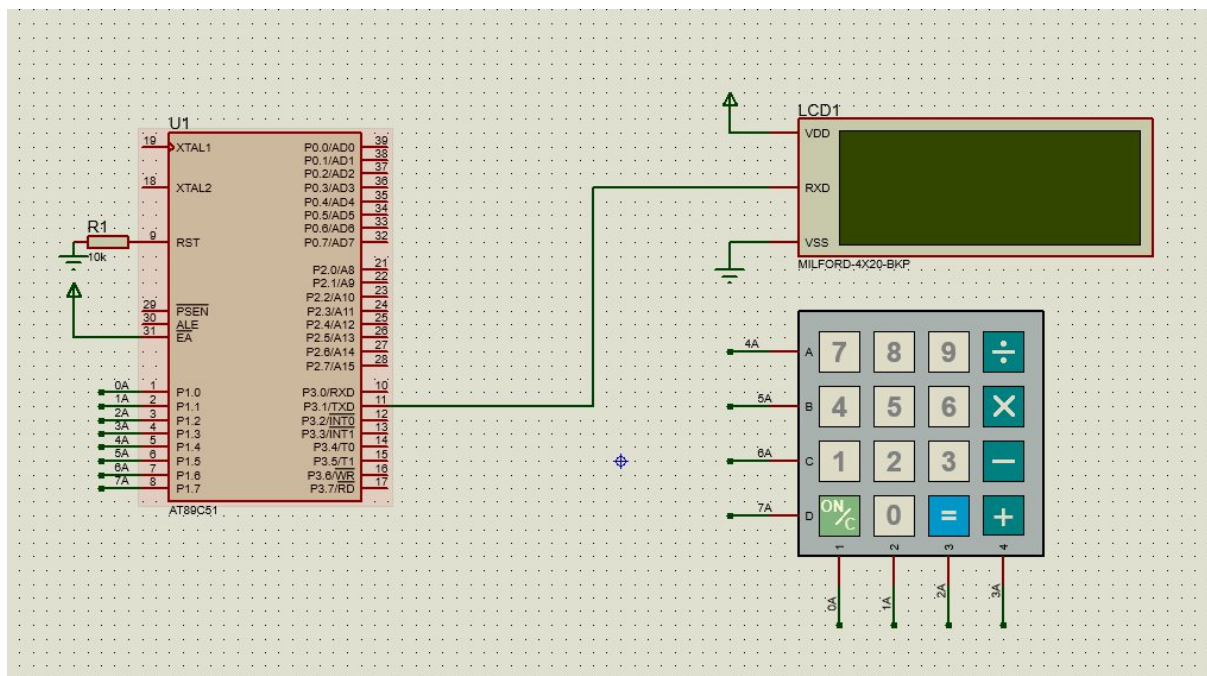
Student's name: Văn Nhật Tân

Student code: 24521587

## PRACTICE EXERCISE #4:

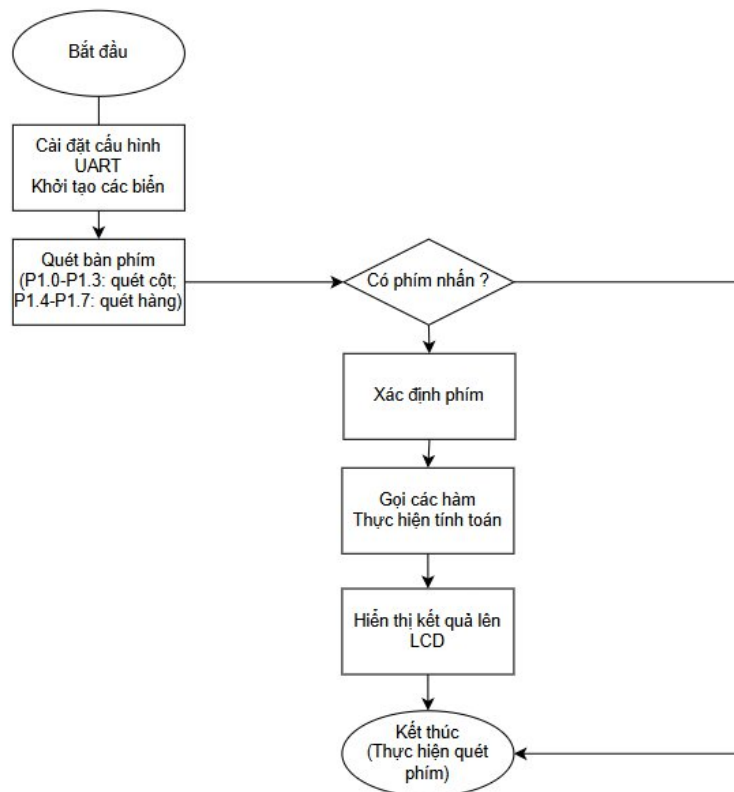
### USING UART

1. Thiết kế máy tính đơn giản phạm vi từ 0 – 9, hiển thị kết quả lên LCD thông qua giao tiếp UART.
  - a. Hình ảnh sơ đồ thiết kế.



- b. Nguyên lý hoạt động mạch:

Flowchart:



Đọc và xử lý phím:

- Thực hiện quét bàn phím theo ma trận 4x4:
  - Xuất tín hiệu theo từng hàng.
  - Đọc tín hiệu phản hồi ở các cột.
  - Khi phát hiện phím nhấn, xác định vị trí và tra cứu giá trị của phím theo cài đặt trong code.

Phân loại phím nhấn:

- Nếu là chữ số (0–9):
  - Nếu flag = 0 → lưu vào num1.
  - Nếu flag = 1 → lưu vào num2.
- Nếu là toán tử (+, -, x, /):
  - Lưu vào op.
  - Đặt flag = 1 → chuyển sang nhập num2.
- Nếu là =:
  - Tiến hành thực hiện phép toán.
  - Hiển thị kết quả lên LCD.
- Nếu là C (Clear):
  - Gửi lệnh xóa màn hình tới LCD.

- Reset lại các biến về trạng thái ban đầu.

### c. Source Code

- Cấu hình UART để giao tiếp LCD với baud rate 9600 bps.

| ASSEMBLY                                                                                                                            | Giải Thích                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> ORG 0000H JMP  START  KEYPAD EQU P1 NUM1 EQU R0 NUM2 EQU R1 OP EQU R2 FLAG EQU R3 TMP EQU R6 </pre>                           | <p><b>ORG 0000H, JMP START:</b> Khởi tạo địa chỉ bắt đầu của chương trình tại 0000H và thực hiện lệnh nhảy đến chương trình chính (START).</p> <p><b>KEYPAD EQU P1:</b> Định nghĩa Port1 là cổng giao với keypad.</p> <p><b>NUM1 EQU R0 và NUM2 EQU R1:</b> Cấp phát thanh ghi R0 và R1 để lưu trữ giá trị của hai toán hạng trong phép tính.</p> <p><b>OP EQU R2:</b> Cấp phát thanh ghi R2 để lưu trữ mã toán tử của phép tính (Cộng, Trừ, Nhân, Chia).</p> <p><b>FLAG EQU R3:</b> Định nghĩa thanh ghi R3 làm cờ trạng thái để quản lý tiến trình nhập liệu của người dùng.</p> <p><b>TMP EQU R6:</b> Cấp phát thanh ghi R6 làm biến tạm để lưu trữ các giá trị tính toán trung gian.</p> |
| <pre> START: MOV  TMOD, #20H MOV  TH1, #0FDH MOV  SCON, #50H SETB TR1  MOV  NUM1, #0 MOV  NUM2, #0 MOV  FLAG, #0 MOV  OP, #0 </pre> | <p><b>MOV TMOD, #20H:</b> Cấu hình Timer 1 - chế độ 2 được sử dụng để làm bộ tạo tốc độ Baud cho giao tiếp nối tiếp UART.</p> <p><b>MOV TH1, #0FDH:</b> Nạp giá trị vào thanh ghi TH1 để thiết lập tốc độ Baud ở mức định mức 9600.</p> <p><b>MOV SCON, #50H:</b> Thiết lập cấu hình thanh ghi điều khiển ở Chế độ</p>                                                                                                                                                                                                                                                                                                                                                                       |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>MOV    TMP, #0</pre>                                                                                                                                                                                                                                                                                                                                                                                                                              | <p>1 và kích hoạt bit cho phép nhận dữ liệu.</p> <p><b>SETB TR1:</b> Timer 1 bắt đầu hoạt động để cấp xung nhịp cho module UART.</p> <p><b>MOV NUM1, #0 đến MOV TMP, #0:</b> Khởi tạo tất cả các biến toán hạng (NUM1, NUM2), toán tử (OP), cờ trạng thái (FLAG) và biến tạm (TMP) về giá trị 0.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <pre> READ_KEYPAD:     MOV    KEYPAD, #0FH BUTTON_WAIT:     MOV    A, KEYPAD     ANL    A, #0FH     CJNE   A, #0FH, PUSHED_BUTTON     SJMP    BUTTON_WAIT  PUSHED_BUTTON:     LCALL   DELAY     MOV    A, KEYPAD     ANL    A, #0FH     CJNE   A, #0FH, FIND_ROW     SJMP    READ_KEYPAD  FIND_ROW:     MOV    KEYPAD, #0EFH     MOV    A, KEYPAD     ANL    A, #0FH     CJNE   A, #0FH, SCAN_R1      MOV    KEYPAD, #0DFH     MOV    A, KEYPAD </pre> | <p><b>READ_KEYPAD</b> và <b>BUTTON_WAIT:</b> Cấu hình giá trị ban đầu cho ma trận phím (0FH - đưa các hàng xuống mức 0, các cột lên mức 1). Vòng lặp <b>BUTTON_WAIT</b> liên tục đọc trạng thái các cột để phát hiện có phím được nhấn hay không.</p> <p><b>PUSHED_BUTTON:</b> Gọi hàm trễ để thực hiện chống dôi phím bằng phần mềm, sau đó xác nhận lại trạng thái phím để loại bỏ nhiễu tín hiệu.</p> <p><b>FIND_ROW:</b> Thực hiện thuật toán quét ma trận. Lần lượt đưa từng hàng xuống mức logic 0 và đọc lại các cột. Nếu phát hiện cột xuống mức 0, chương trình xác định được hàng đang được nhấn.</p> <p><b>SCAN_R1 đến SCAN_R4:</b> Nạp địa chỉ bắt đầu của mảng dữ liệu chứa mã ASCII của hàng tương ứng vào con trỏ dữ liệu (DPTR).</p> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> ANL    A, #0FH CJNE   A, #0FH, SCAN_R2  MOV     KEYPAD, #0BFH MOV     A, KEYPAD ANL     A, #0FH CJNE    A, #0FH, SCAN_R3  MOV     KEYPAD, #7FH MOV     A, KEYPAD ANL     A, #0FH CJNE    A, #0FH, SCAN_R4  LJMP    READ_KEYPAD  SCAN_R1: MOV     DPTR, #ROW1 SJMP    SCAN SCAN_R2: MOV     DPTR, #ROW2 SJMP    SCAN SCAN_R3: MOV     DPTR, #ROW3 SJMP    SCAN SCAN_R4: MOV     DPTR, #ROW4 SJMP    SCAN  SCAN: RRC     A JNC     MATCH INC     DPTR SJMP    SCAN  MATCH: </pre> | <p><b>SCAN:</b> Sử dụng lệnh dịch phải qua cờ Carry để tìm chính xác vị trí cột đang được nhấn. Mỗi lần dịch, con trỏ DPTR được tăng lên 1 đơn vị để trỏ đúng vào ký tự tương ứng.</p> <p><b>MATCH:</b> Truy xuất dữ liệu (mã ASCII của phím) từ bộ nhớ ROM dựa trên địa chỉ đã xác định ở DPTR, sau đó gọi hàm CALCULATE để xử lý logic toán học.</p> <p><b>WAIT_RELEASE:</b> Vòng lặp chờ nhả phím. Giữ hệ thống chờ cho đến khi người dùng thả tay khỏi phím bấm nhằm đảm bảo mỗi lần nhấn chỉ ghi nhận dữ liệu một lần, chống hiện tượng trôi phím.</p> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> MOV    A, #0 MOVC   A, @A+DPTR LCALL  CALCULATE  WAIT_RELEASE: MOV    KEYPAD, #0FH MOV    A, KEYPAD ANL    A, #0FH CJNE   A, #0FH, WAIT_RELEASE LJMP   READ_KEYPADI </pre>                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <pre> CALCULATE: CJNE   A, #'C', CHECK_EQUAL LJMP   CLEARLCD  CHECK_EQUAL: LCALL  DISPLAY CJNE   A, #'=', NOTEQUAL MOV    A, OP  CJNE   A, #'+', CHECK_SUB MOV    A, NUM1 ADD    A, NUM2 SJMP   DONE_CALCULATE  CHECK_SUB: CJNE   A, #'-', CHECK_MUL CLR    C MOV    A, NUM1 SUBB   A, NUM2 JC     PRINT_NEGATIVE LJMP   DONE_CALCULATE  PRINT_NEGATIVE: MOV    B, A MOV    A, #'-' </pre> | <p><b>CALCULATE</b> và <b>CHECK_EQUAL</b>: Kiểm tra phím người dùng vừa nhấn. Nếu là phím xóa 'C', lệnh nhảy đến hàm xóa màn hình. Nếu là dấu '=', chương trình sẽ lấy toán tử đã lưu (trong thanh ghi OP) nạp vào thanh ghi A để rẽ nhánh đến phép tính tương ứng.</p> <p><b>Xử lý phép Cộng</b>: Kiểm tra nếu toán tử là '+', thực hiện lệnh ADD A, NUM2 (cộng NUM1 và NUM2).</p> <p><b>CHECK_SUB</b> và <b>PRINT_NEGATIVE (Phép Trừ)</b>: Xóa cờ nhớ và thực hiện trừ NUM1 cho NUM2. Lệnh JC kiểm tra xem có xảy ra mượn hay không nếu có, nhánh PRINT_NEGATIVE sẽ in dấu trừ '-' ra màn hình trước, rồi chuyển kết quả từ dạng bù 2 về dạng số tuyệt đối để hiển thị.</p> <p><b>CHECK_MUL (Phép Nhân)</b>: Sử dụng lệnh MUL AB để nhân phần cứng hai thanh ghi chứa toán hạng.</p> <p><b>CHECK_DIV, DO_DIV,</b></p> |

|                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> LCALL DISPLAY MOV  A, B CPL  A INC  A SJMP DONE_CALCULATE  CHECK_MUL: CJNE A, #'X', CHECK_DIV MOV  A, NUM1 MOV  B, NUM2 MUL  AB SJMP DONE_CALCULATE  CHECK_DIV: MOV  A, NUM2 CJNE A, #0, DO_DIV LJMP DIVIDE_ERROR  DO_DIV: MOV  A, NUM1 MOV  B, NUM2 DIV  AB MOV  TMP, B LCALL PRINT_NUMBER MOV  A, #'.' LCALL DISPLAY  MOV  A, TMP MOV  B, #10 MUL  AB MOV  B, NUM2 DIV  AB MOV  TMP, B LCALL PRINT_NUMBER </pre> | <p><b>CHECK_GREATER:</b> Thực hiện kiểm tra lỗi chia 0, nếu số chia là 0 thì nhảy đến hàm DIV_ERROR, ngược lại nhảy đến hàm DO_DIV. Lệnh chia DIV AB. Phần nguyên được in ra trước, sau đó in dấu chấm thập phân. Tiếp tục lặp lại quá trình nhân số dư với 10 rồi chia tiếp để có 2 chữ số thập phân kèm theo điều kiện kiểm tra (CJNE A, #5) để làm tròn lên nếu giá trị phần thập phân tiếp theo lớn hơn hoặc bằng 5.</p> <p><b>DONE_CALCULATE:</b> Gọi hàm PRINT_NUMBER để xuất kết quả cuối cùng từ thanh ghi A ra màn hình, đồng thời đặt lại cờ trạng thái (FLAG = 0) để dọn dẹp hệ thống, chuẩn bị nhận phép tính mới.</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> MOV    A, TMP MOV    B, #10 MUL    AB MOV    B, NUM2 DIV    AB CJNE   A, #5, CHECK_GREATER INC    A SJMP   DONE_CALCULATE  CHECK_GREATER: JC     DONE_CALCULATE INC    A  DIVIDE_ERROR: MOV    A, #'E' LCALL  DISPLAY MOV    A, #'R' LCALL  DISPLAY MOV    A, #'R' LCALL  DISPLAY MOV    FLAG, #0 RET  DONE_CALCULATE: LCALL  PRINT_NUMBER MOV    FLAG, #0 RET </pre> |                                                                                                                                                                                                                                                                                          |
| <pre> NOTEQUAL: CJNE   A, #'+', CHECK_OPT2 MOV    OP, A MOV    FLAG, #1 RET  CHECK_OPT2: CJNE   A, #'-', CHECK_OPT3 MOV    OP, A MOV    FLAG, #1 </pre>                                                                                                                                                                                                                     | <p><b>NOTEQUAL đến</b><br/> <b>CHECK_OPT4:</b> Kiểm tra xem phím vừa nhấn có phải là các phép toán (+, -, x, /) hay không. Nếu đúng, chương trình lưu mã phép toán vào thanh ghi OP và thiết lập cờ FLAG = 1 để báo hiệu hệ thống chuyển sang trạng thái chờ nhập toán hạng thứ hai.</p> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> RET CHECK_OPT3:     CJNE  A, #'X', CHECK_OPT4     MOV   OP, A     MOV   FLAG, #1     RET  CHECK_OPT4:     CJNE  A, # '/', STORE_DIGIT     MOV   OP, A     MOV   FLAG, #1     RET  STORE_DIGIT:     MOV   B, A     MOV   A, FLAG     CJNE  A, #0, CIN_NUM2     CLR   C     MOV   A, B     SUBB  A, #'0'     MOV   NUM1, A     RET  CIN_NUM2:     MOV   A, B     CLR   C     SUBB  A, #'0'     MOV   NUM2, A     RET  PRINT_NUMBER:     PUSH  B     MOV   B, #10     DIV   AB     CJNE  A, #0, PRINT_TENS     SJMP  PRINT_UNITS  PRINT_TENS: </pre> | <p><b>STORE_DIGIT và CIN_NUM2:</b><br/>         Nếu phím nhấn không phải là toán tử, mặc định đó là chữ số (0-9). Dựa vào trạng thái của cờ FLAG, chuyển đổi mã ASCII thành giá trị số và lưu vào NUM1 (nếu FLAG = 0) hoặc NUM2 (nếu FLAG = 1).</p> <p><b>PRINT_NUMBER:</b> Chuyển đổi số sang mã ASCII để hiển thị. Sử dụng phép chia cho 10 để tách lấy hàng chục và hàng đơn vị sử dụng thêm cặp lệnh PUSH B và POP B để cất giữ và phục hồi giá trị thanh ghi B, ngăn không cho quá trình hiển thị làm hỏng các phép tính toán học đang diễn ra.</p> <p><b>DISPLAY:</b> Thực hiện đẩy dữ liệu từ thanh ghi A vào bộ đệm truyền nối tiếp (SBUF) để xuất tín hiệu TXD ra màn hình LCD.</p> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> ADD  A, #'0' LCALL DISPLAY PRINT_UNITS: MOV  A, B ADD  A, #'0' LCALL DISPLAY POP  B RET  CLEARLCD: LCALL DELAY MOV  A, #0FEH LCALL DISPLAY LCALL DELAY MOV  A, #01H LCALL DISPLAY LJMP START  DISPLAY: MOV  SBUF, A </pre> |                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <pre> WAITTI: JNB  TI, WAITTI CLR  TI RET  DELAY: MOV  R4, #136 LOOP1: MOV  R5, #100 LOOP2: DJNZ R5, LOOP2 DJNZ R4, LOOP1 RET </pre>                                                                                             | <p><b>WAITTI:</b> Hàm chờ cờ ngắt. Sau đó, chương trình thực hiện lệnh CLR TI để xóa cờ, chuẩn bị cho lần truyền dữ liệu tiếp theo.</p> <p><b>DELAY:</b> Tạo trễ chống dội phím.</p> <p><b>ORG 500H và cụm ROW1 đến ROW4:</b> Lệnh ORG 500H chỉ định trình biên dịch lưu trữ khối dữ liệu tiếp theo bắt đầu từ địa chỉ 500H trong bộ nhớ ROM, chứa mã ASCII của các ký tự tương ứng với cấu trúc vật lý của bàn phím ma trận 4x4.</p> |

|                                                                                                                                     |  |
|-------------------------------------------------------------------------------------------------------------------------------------|--|
| ORG 500H<br>ROW1: DB '7','8','9','/'<br>ROW2: DB '4','5','6','X'<br>ROW3: DB '1','2','3','-'<br>ROW4: DB 'C','0','=','+'<br><br>END |  |
|-------------------------------------------------------------------------------------------------------------------------------------|--|

## 2. Video mô phỏng hoạt động của mạch:

<https://drive.google.com/drive/folders/1scz3VBM1Yjh7WqltNpwDWQVrwXkqqRbw?usp=sharing>